

BBK2HAB3

Enterprise Application Integration

Week 09

Introduction to Microservices

Presented By:

Purnama Anaking, S.Kom., M.Kom.



Outline

- Introduction to Microservices
- Container Tools for Microservices
- Docker for Microservices
- Kubernetes for Microservices Orchestration
- Microservices Application Implementation and Deployment

Why are Microservices Important?

- **Complexity of Modern Applications:** Applications now not only display data, but also handle many functions such as login, transactions, notifications, analytics, etc.
- **Specific Scalability:** In e-commerce applications, for example: the catalog module can be very busy during promotions, but the admin panel may not.
- **Faster and Safer Changes:** Teams can release new features without disrupting other systems.
- **Technology Flexibility:** Each service can use the programming language, framework, or database that is best suited for its task. For example, Python for machine learning services, Node.js for API Gateway.
- **Supports DevOps and CI/CD:** Microservices are suitable for automated pipelines. Deployments can be done more frequently and in a controlled manner (for example: rolling updates per service).

Monolith vs Microservice

- **Monolithic** is like a big store with one entrance and one cashier. If the cashier has a problem, all the store services are disrupted.
- **Microservices** is like a shopping mall with many different stores. Each store has its own cashier and services. If one store closes, the others keep running.

Characteristics

- Service independence
- Communication via API
- Scale of each service

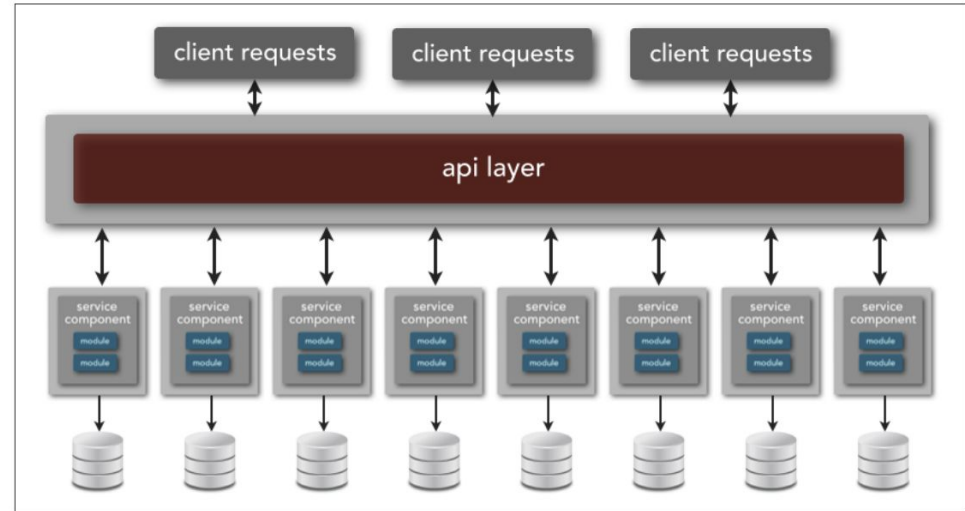
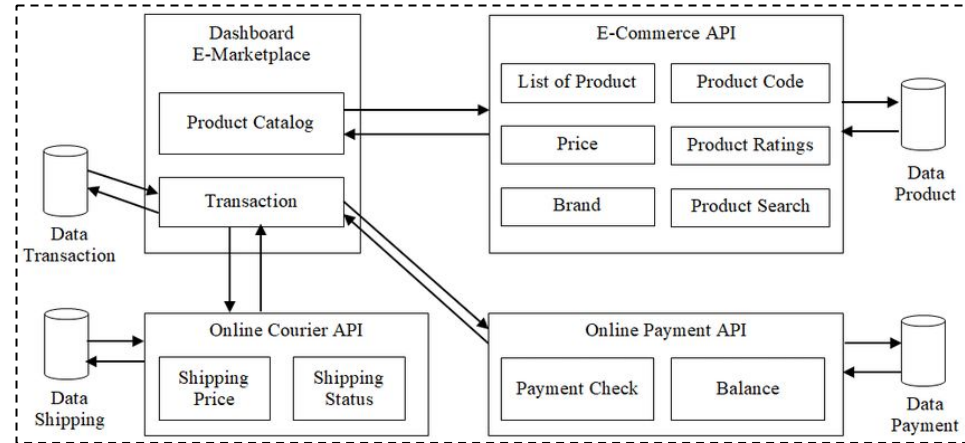


Figure 3-5. Microservices architecture topology

Common Components

- Service Registry
- API Gateway
- Service-to-service communication



What is Service Registry ?

Before knowing about the registry service, first know what a registry is.

What is a Registry?

- A **registry** is a central repository for storing, managing, and distributing certain digital objects, such as Docker images, service configurations, or service metadata.

Two Common Contexts of "Registry":

- **1. Container Image Registry:** A place to store Docker images or container images. Used when we docker push (upload images) and docker pull (get images).
 - ◆ **Analogy:** A container image registry is like the App Store but for applications in container form. Developers upload applications (images), and anyone can download and run them.
- **2. Service Registry:** A place where services (microservices) register themselves so that they can be found by other services. Used in service discovery.
 - ◆ **Analogy:** A service registry is like a digital address book or online receptionist for applications. If you want to "contact a payment service", you ask the registry, not guess the address.

Registry Summary

- A **registry** is a central repository for storing, managing, and distributing certain digital objects, such as Docker images, service configurations, or service metadata.

Registry Types	Function	Example
Container Registry	Store and distribute container images	Docker Hub, GCR, ECR
Service Registry	Central directory for service discovery	Eureka, Consul, etc

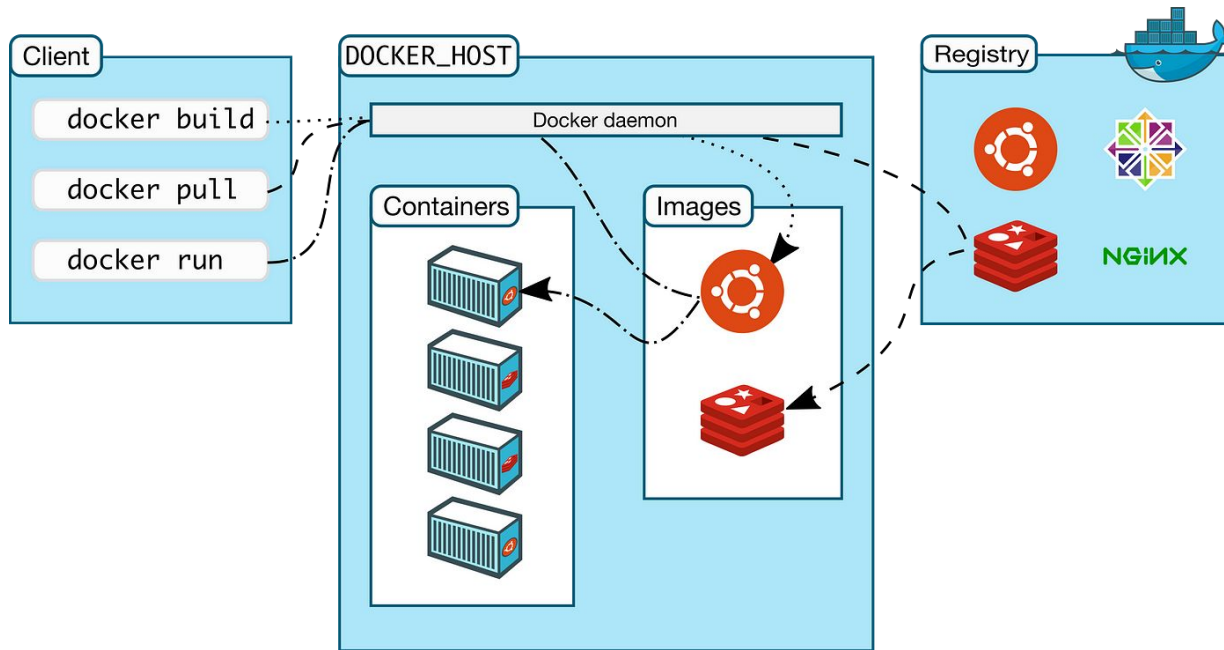
Service Registry

- **Service Registry** is an important component in microservices architecture that serves as a central directory where all services register themselves and can be discovered by other services.

Main Functions:

- **Registration:** Every running microservice will register its information (name, IP address, port, status) to the registry.
- **Discovery:** Microservices that need other services will find out the location of the service through the service registry.
- **Health Check:** The registry periodically checks whether the registered services are still active and deletes those that are not responding.

Registry Overview



Container Tools for Microservices

What is a Container?

- A **container** is a standard unit of software that encapsulates code and all its dependencies so that applications can run quickly and consistently across environments.

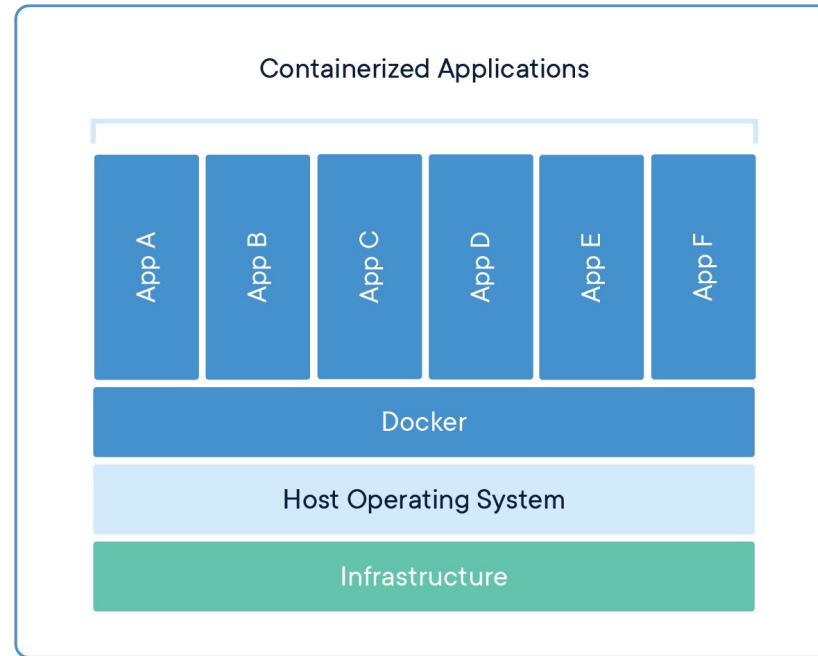
Benefits of Containers in Microservices:

- **Isolation:** Each service can run in its own environment.
- **Portability:** Can be run on multiple systems (development, staging, production).
- **Consistency:** “It works on my machine” is no longer a problem.

Container Tools for Microservices

- **Virtual Machine (VM)** is an abstraction of physical hardware that allows a single server to run multiple virtual systems through a hypervisor. Each VM requires its own operating system installation, which can result in large application sizes and increased resource consumption, especially when using multiple VMs (can be more than 1 GB per VM).
- **Container** is an application-level abstraction that packages code and dependencies into a single package. Multiple containers can run on a single machine, share an OS kernel, and run as isolated processes through a container manager. They are lighter than VMs, typically only tens of MB.

Containers for Microservices



Introduction to Docker

- Docker is an open source platform for **building, running, and distributing** applications in **containers**.
- Docker **decouples** applications from **infrastructure**, enabling faster and more consistent deployment **across environments**, such as **local, cloud, and production**.
- Docker containers package **source code, runtime, and dependencies**, and **run** applications in isolation **without disrupting** the host system.

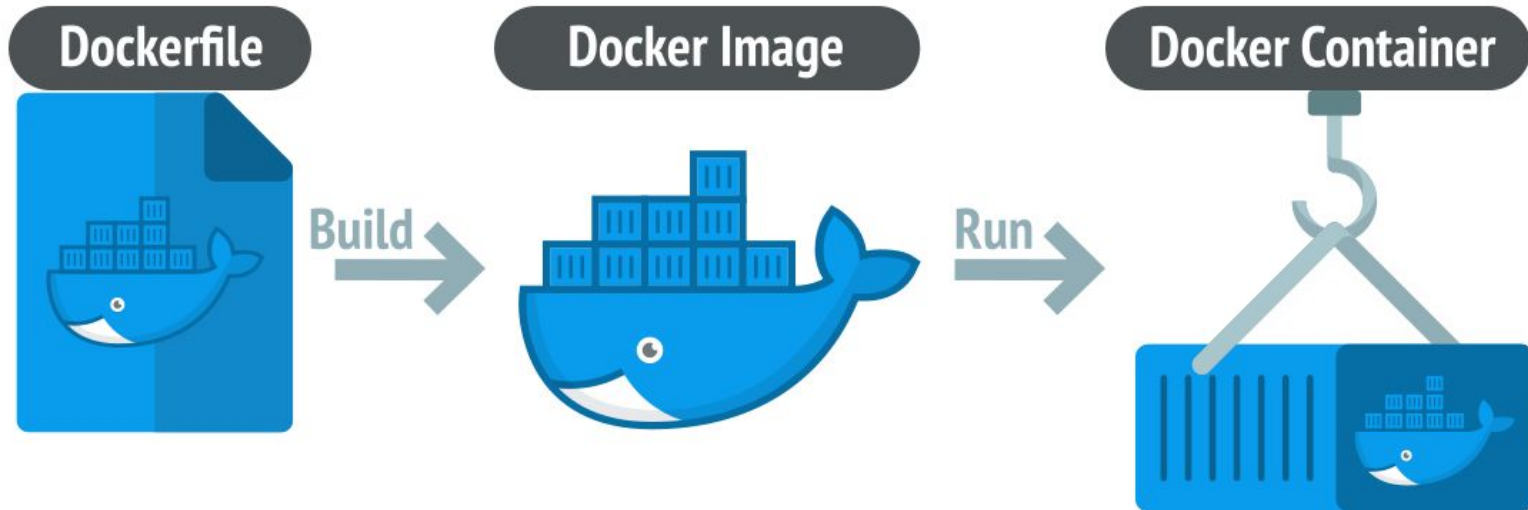
Key Docker Components

- **Image:** Read-only template for containers.
- **Container:** Runnable instance of an image.
- **Dockerfile:** Configuration file for building Docker images.
- **Docker Hub:** Public image repository.

Advantages of Docker in Microservices:

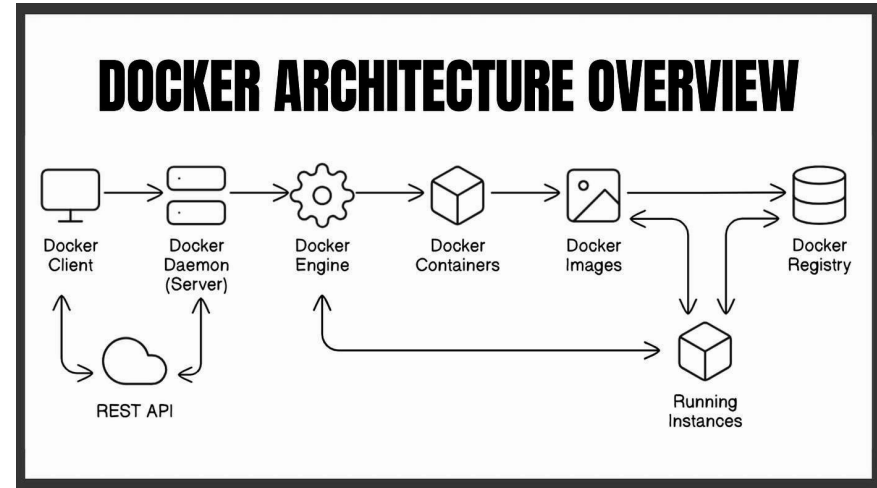
- **Fast and lightweight:** More efficient than virtual machines.
- **Repeatable builds:** Applications are built from replicable definitions.
- **CI/CD support:** Perfect for modern development pipelines.
- **Integration with other tools:** Such as Jenkins, Kubernetes, and others.

Docker Container

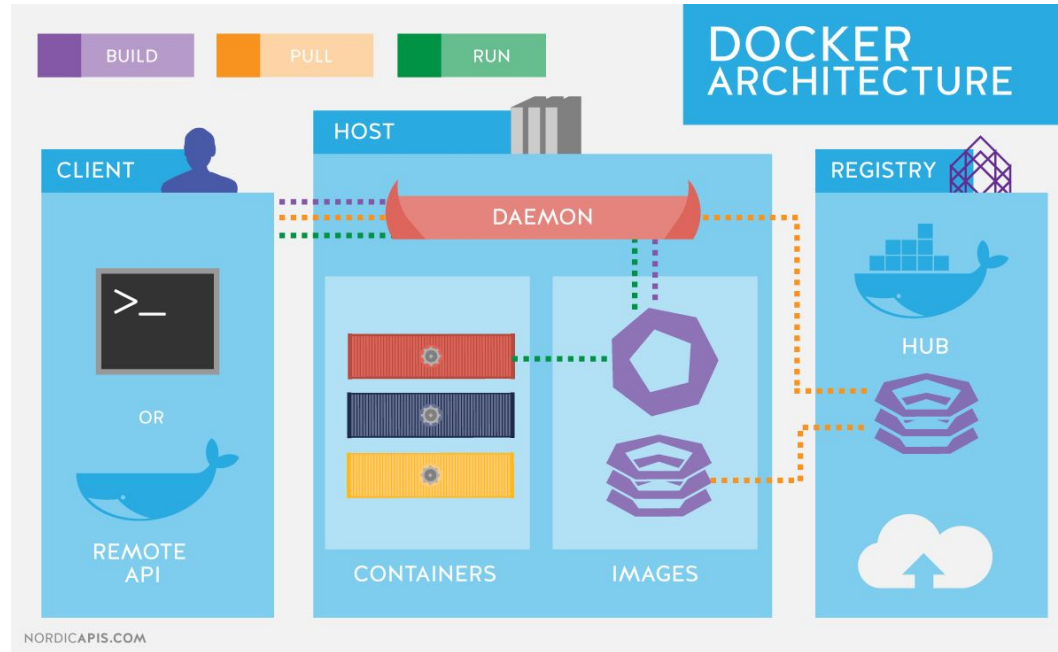


Docker Architecture Overview

- Docker uses a client-server architecture, where the Docker client sends commands to the Docker daemon to build, run, and distribute containers.
- Both can run on the same system or be connected over a network.
- Communication between components is done through a REST API.
- Another client is Docker Compose, which is used to handle multi-container applications.



Docker Architecture



Docker Architecture

- **Docker Daemon** listens to Docker API requests and manages Docker objects such as images, containers, networks, and volumes. A Docker daemon can also communicate with other daemons to manage docker services.
- **The Docker Client** is a user interface for interacting with Docker. When using a command such as **docker run**, the client sends this command to dockerd, which runs it. Docker commands use the Docker API. The Docker Client can communicate with more than one daemon.
- **Docker Desktop** is a Docker application installed on a Mac, Windows, or Linux environment that allows you to build and share containerized applications and microservices. Docker desktop includes the Docker daemon (dockerd), Docker client (docker), Docker compose, Docker content trust, kubernetes, and credential helper.

Docker Architecture

- **The Docker Registry** is a place to store Docker Images. By using the Docker Registry, we can store the Images we create, and can be used in the Docker Daemon anywhere as long as it can connect to the Docker Registry. When using the `docker pull` or `docker run` command, Docker pulls the required image from a configured registry. When using the `docker push` command, Docker pushes the image to a configured registry.
- **Docker Objects** are components found in Docker. Docker Objects can include images, containers, networks, volumes, and other objects.

Kubernetes for Microservices Orchestration

- **Kubernetes** is an open-source **container orchestration** platform designed to **automate the deployment, scaling,** and management of **container-based** applications.



kubernetes

Introduction to Kubernetes

- **Orchestration Functions:** Container scheduling, Monitoring and auto-healing, Auto-scaling, Load balancing.

Main Kubernetes Components:

- **Pod:** The smallest unit in Kubernetes, can contain one or more containers.
- **Deployment:** To manage the number of replicas, update strategies, and rollbacks.
- **Service:** To expose applications in the pod to the network.
- **ConfigMap and Secret:** For dynamic configuration.
- **Ingress:** For managing access from outside to inside the cluster.

Implementation and Deployment of Microservices Applications with Docker and Kubernetes

Implementation Flow:

- Separate application functions into several services (example: user-service, product-service, order-service).
- Build **Dockerfile** for each service.
- Test locally with **Docker Compose**.
- Deploy to Kubernetes with **YAML** files:
 - ◆ **Deployment.yaml** to define pods.
 - ◆ **Service.yaml** for expose.
 - ◆ **Ingress.yaml** for routing.

Conclusion

- **Microservices** = system modularization
- **Docker** = service packaging and isolation
- **Kubernetes** = service orchestration and management
- This combination improves application **scalability, portability, and reliability.**

Active Learning

- Buat project UTS yang lalu dengan pendekatan microservices menggunakan **Docker**.
- Terapkan **komunikasi asinkron** menggunakan teknologi seperti **Laravel Queue, Redis** atau **RabbitMQ**.
- **Challenges:** terapkan **Kubernetes** pada project tersebut.
- Berkelompok sesuai kelompok UTS
- Deadline: 1 minggu.
- Buat video yang mendemokan dan menjelaskan apa yang telah dilakukan. Buat file **note.txt** yang berisi **Nama Kelompok, Nama anggota, NIM, link video**.

Any Questions?
